A Mixed-Integer Optimization Approach for Tangram Puzzles

Yossi Daniel[0009−0007−5029−1896] and Hillel Kugler[0000−0001−7924−5665]

Abstract Two-dimensional (2D) polygonal tiling presents challenging optimization tasks due to the combinatorial explosion of valid arrangements. We introduce a mixed-integer second-order cone programming (MI-SOCP) framework that maps the continuous search space to a discrete lattice grid, producing initial approximate solutions with grid-restricted translations. A final fine-tuning optimization (FTO) stage then refines these discrete translations into continuous coordinates. To ensure computational tractability over large variable spaces, an efficient geometric preprocessing pipeline prunes symmetric and redundant topological states prior to optimization. Experiments on benchmark Tangram puzzles demonstrate that this approach efficiently yields high-quality approximate solutions, which are subsequently refined into continuous-coordinate configurations that satisfy a strict numerical geometric criterion.

Keywords: Mixed-integer constrained optimization, 2D tiling, Tangram, Space discretization, Sutherland-Hodgman (SH) algorithm.

1 Introduction

A finite 2D Tangram problem involves covering a target geometry using a given set of polygonal tiles without overlap, utilizing translation, rotation, and reflection. Although intuitively approachable for humans, automated solution methods must navigate a combinatorial explosion within high-dimensional configuration spaces. We introduce a mixed-integer optimization framework to mathematically model and solve these spatial configurations, validating our approach on classical benchmark Tangram puzzles [4, 6, 13]. The source code and experimental results are made publicly available at [2].

Yossi Daniel and Hillel Kugler
Faculty of Engineering, Bar-Ilan University, Ramat Gan, Israel, e-mail: danielyd@biu.ac.il, hillelk@biu.ac.il

Key Contributions. We position our framework against existing paradigms across two axes: (1) **Search Heuristics vs. Mixed-Integer Optimization:** Unlike meta-heuristics, e.g., Simulated Annealing (SA) and Genetic Algorithms (GA), that explore continuous spaces without exploiting strict geometric constraints, often suffering from poor convergence to global minima, our approach converges to precise solutions with a high success rate. (2) **Learning-Based vs. Mathematical Programming:** In contrast to data-intensive deep learning approaches that require heavy computational overhead for training, our MI-SOCP paradigm is training-free. By utilizing a one-time offline geometric preprocessing phase, it transforms the layout domain into a highly tractable optimization model. Empirically, our method is effective in solving standard Tangram benchmarks and is naturally extensible to heterogeneous polygons, imperfect covers, and higher-dimensional packing.

2 Related Work

Finite 2D tiling and packing are well-studied via combinatorial search and mathematical programming.

Combinatorial and Heuristic Search: Tiling is a challenging problem, many packing variants are NP-complete [3, 11], and tiling simply connected regions with rectangles involves intricate constraints [14]. Grid-based puzzles (e.g., polyominoes [7]) are often framed as exact cover problems solved via Knuth's Algorithm X [12]. However, continuous puzzles like Tangrams typically require heuristic or meta-heuristic search methods [3].

Modern Computational Approaches: Recent developments leverage raster-based mathematical morphology for spatial constraints [19] or generative AI models [17, 18]. Additionally, Tangram puzzles serve as key benchmarks for evaluating machine spatial reasoning and visual task learning [9, 20].

Mathematical Optimization: In operations research, 2D tiling intersects with irregular strip packing or nesting [1, 16]. Mixed-Integer Programming (MIP) is standard for orthogonal bin packing, but extending it to arbitrary polygons with rotational freedom introduces non-convex overlap constraints that are computationally expensive [5]. Our approach differs by using a spatial discretization pipeline to recast the problem as a highly structured MI-SOCP model.

3 Problem Formulation

While the Tangram problem is inherently defined over a continuous geometric space, our framework maps this domain onto a discrete lattice grid to enable mathematical programming. This transforms the layout task into a constrained mixed-integer optimization model that seeks the optimal reflection, rotation, and discrete translation for

each tile. A final fine-tuning optimization (FTO) stage then refines these grid-aligned translations into continuous coordinates. An overview of this integrated pipeline is illustrated in Figure 1.

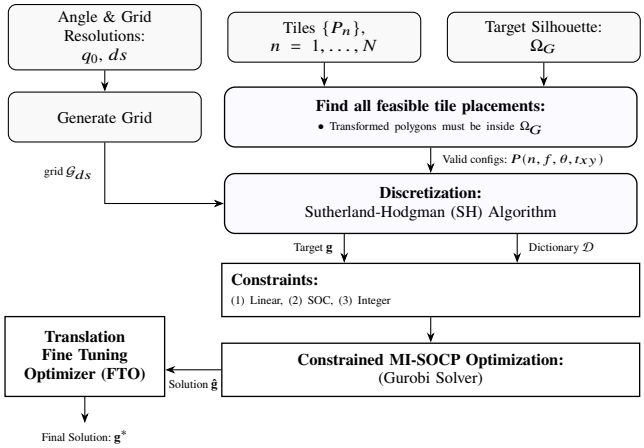


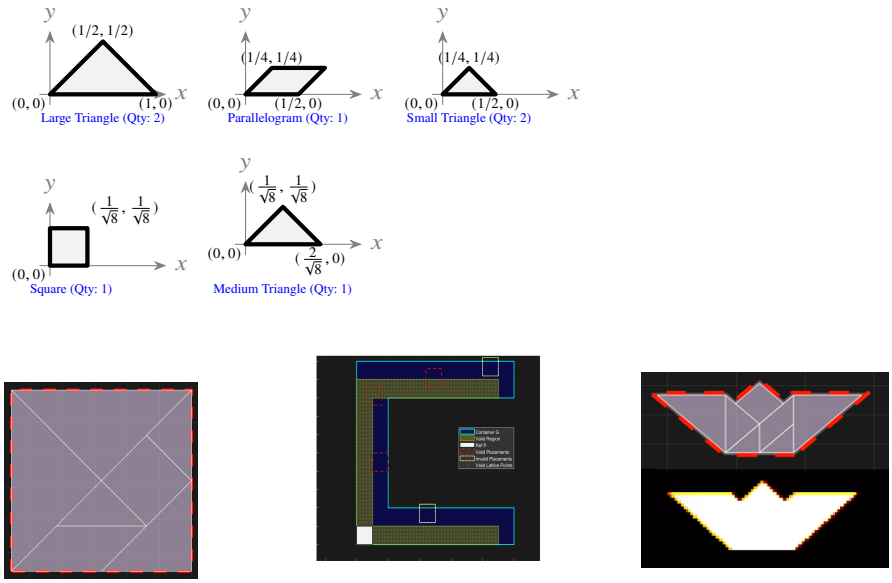
Fig. 1 Overview of the MI-SOCP pipeline (lattice mapping, optimization, and FTO). The FTO *exclusively optimizes translations* to compensate for the discrete grid, keeping flips and rotations fixed.

Continuous Mathematical Formulation: Let $\Omega_G \subset \mathbb{R}^2$ be a compact target silhouette geometry (represented by a list of outer ring vertices plus a list of vertices per hole), and $\mathcal{M} = \{1, \ldots, N\}$ be the index set of N polygonal tiles. Each tile $n \in \mathcal{M}$ is defined as a closed, compact polygon $P_n \subset \mathbb{R}^2$ initialized at its canonical origin ($\theta = 0°, f = 0, \mathbf{t}_{xy} = [0, 0]^\top$). We establish a bounding box $\mathcal{H} = \{(x, y) \in \mathbb{R}^2 \mid -x_\ell \le x \le x_r, -y_b \le y \le y_t\}$ chosen such that $\Omega_G \subset \mathcal{H}$ (with appropriate padding margins). A rigid transformation operator $\mathcal{T}_n$ maps each tile n to a placement $\tilde{P}_n \subset \mathcal{H}$ using a continuous translation $\mathbf{t}_{xy} \in \mathbb{R}^2$, rotation $\theta \in [0, 2\pi)$, and binary reflection indicator $f \in \{0, 1\}$:

$$\tilde{P}_n = \mathcal{T}_n(P_n; \mathbf{t}_{xy}, \theta, f) = \{\mathbf{M}(\theta, f)\mathbf{x} + \mathbf{t}_{xy} \mid \mathbf{x} \in P_n\}, \tag{1}$$

where $\mathbf{M}(\theta, f) = \mathbf{R}(\theta)\mathbf{F}(f)$. The explicit individual component transformations are given by:

$$\mathbf{R}(\theta) = \begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix}, \quad \mathbf{F}(f) = \begin{bmatrix} (-1)^f & 0 \\ 0 & 1 \end{bmatrix}, \quad \mathbf{t}_{xy} = \begin{bmatrix} t_x \\ t_y \end{bmatrix}. \tag{2}$$



(a) Decomposition (b) Minkowski subset. (c) Lattice mapping.

Fig. 2 Geometric definitions (top), Decomposition, Minkowski subset and lattice mapping (using SH) pipeline example (bottom).

An exact continuous solution requires finding transformations $\{\mathcal{T}_n\}_{n=1}^{N}$ satisfying: (1) **Perfect Coverage:** $\bigcup_{n=1}^{N} \tilde{P}_n = \Omega_G$, and (2) **Non-Overlap Condition:** [1] $\text{int}(\tilde{P}_n) \cap \text{int}(\tilde{P}_m) = \emptyset,\ \forall n, m \in \mathcal{M},\ n \neq m$, where $\text{int}(\cdot)$ denotes the polygon interior operator. Geometric vertex definitions for the Tangram set are detailed in Fig. 2 (top), and a decomposition example for a square target is shown in Figure 2a.

Space Discretization: We project $\mathcal{H}$ onto a lattice grid $\mathcal{G}_{ds}$ with step size ds. We define $H = \text{ceil}((y_b + y_t)/ds)$, $W = \text{ceil}((x_\ell + x_r)/ds)$, and $d = H \cdot W$. Transformations are restricted as follows: translations $\mathbf{t}_{xy}$ are constrained to grid nodes, rotations $\theta \in \{k \cdot 45°\}$, and reflections $f \in \{0, 1\}$. Sutherland-Hodgman (SH) clipping [15] computes the fractional occupancy $\mathbf{v}_n \in [0, 1]^d$ per configuration, ensuring a finite search space that is mapped to binary selection variables.

Dictionary of Feasible Placements: A static dictionary $\mathcal{D}$ is precomputed by admitting any transformed state $\tilde{P}_n$ (Fig. 2) fully contained within the target domain. For Tangrams, feasible translation boundaries T_{feas} are derived via the Minkowski difference:

$$T_{\text{feas}} = \bigcap_{v_i \in \mathcal{T}_n(P_n; \mathbf{0}, \theta, f)} (\Omega_G^{\text{ch}} - v_i), \tag{3}$$

[1] Explicit pairwise non-overlap constraints, while supported by the mathematical model in principle, are not included in the reported MI-SOCP experiments. Instead, overlap of the final continuous configurations is evaluated directly after FTO using the OLA metric defined in Section 4.1.

where Ω_G^{ch} is the goal shape's convex hull. This relaxation prevents the erroneous exclusion of valid translations due to target concavities and holes. False positives introduced by the convex hull are subsequently filtered via exact area inclusion checks.

Symmetry and Discretization Pipeline: Redundant states are pruned: most pieces are flip-agnostic ($f = 0$), and the square is restricted to $\theta \in \{0°, 45°\}$. Configurations $\tilde{P}_n$ are clipped against $\mathcal{G}_{ds}$ via SH. 2D spatial grids are flattened into 1D vectors of size $d = H \cdot W$ via the vectorization operator $\text{vec}(\cdot)$, yielding profiles $\mathbf{g} = \text{vec}(\text{SH}(\Omega_G, \mathcal{G}_{ds}))$ and $\mathbf{v}_n = \text{vec}(\text{SH}(\tilde{P}_n, \mathcal{G}_{ds}))$.

Discretization Sensitivity and Resolution Selection: The step size ds balances geometric precision with optimization complexity. Finer discretization (decreasing ds) minimizes quantization error but induces quadratic expansion in the lattice dimension $d = H \cdot W$, inflating dictionary size and runtime. We select a baseline $ds = 1/28$ (relative to a unit square silhouette edge); this resolution ensures high success rates across benchmarks while remaining fully tractable on standard hardware without non-uniform spatial refinement.

Algorithm 1: Dictionary Generation.

In: $\{P_n\}, \Omega_G, \mathcal{G}_{ds}, ds, q_0$ Out: Partitioned Dictionary $\mathcal{D} = \{\mathcal{D}_1, \ldots, \mathcal{D}_N\}$
Init: $\mathcal{D} \leftarrow \emptyset$
for $n \leftarrow 1$ **to** N **do**
  $\mathcal{D}_n \leftarrow \emptyset$
  for $f \in \{0, 1\}, \theta \in \{k \cdot q_0\}$ **do**
    $P'_n \leftarrow \mathcal{T}_n(P_n; \mathbf{0}, \theta, f)$
    for $\mathbf{t}_{xy} \in$ *CalcFeasible*(P'_n, Ω_G) **do**
      $\mathcal{D}_n \leftarrow \mathcal{D}_n \cup \{\text{vec}(\text{SH}(\text{translate}(P'_n, \mathbf{t}_{xy}), \mathcal{G}_{ds}))\}$
  $\mathcal{D} \leftarrow \mathcal{D} \cup \{\mathcal{D}_n\}$

Alg. 1 constructs partitioned subsets $\mathcal{D}_n$, enabling uniqueness constraints in the MI-SOCP model.

3.1 Fine-Tuning Optimizer (FTO)

The final pipeline stage employs a Fine-Tuning Optimizer (FTO) to refine grid-aligned solutions into continuous coordinates. Applied uniformly across all solver outputs as a post-processing step, the FTO freezes discrete flips and rotations while optimizing translations to compensate for discretization artifacts, serving as a unified refinement mechanism with consistent metrics (see definitions of CA, UCA, OLA, and OSA in Section 4.1). A successful FTO refinement validates the quality of the underlying grid-aligned solver solution. The FTO operates in two sequential stages:

Vertex Matching (Stage 1): Aligns tiles to the target's outer ring vertices. For tile P_n, let (x_g, y_g) denote the closest vertex in target G to vertex (x_p, y_p) of P_n,

yielding the difference vector $\mathbf{w}_{gp} = (x_g - x_p, y_g - y_p)$. If $\|\mathbf{w}_{gp}\| < \text{T\_NORM\_TH}$ (set to $\|[2, 2]\|$), the fine-tuned translation for P_n is set to $\mathbf{w}_{gp}$ and P_n is added to `MatchedTilesSet`.

Continuous Translation Refinement Optimizer (Stage 2): Processes unmatched tiles (omitting those in `MatchedTilesSet`) via MATLAB's `patternsearch()` routine to minimize overlap, uncovered, and outside area penalties.

Motivation: By prioritizing high-probability placements via vertex snapping in Stage 1, the search space for continuous refinement in Stage 2 is significantly reduced. Crucially, achieving successful refinement with minimal UCA, OLA, and OSA metrics via minor translation adjustments provides robust evidence of high solution quality from the underlying solver.

4 Mathematical Optimization Model

The 2D tiling problem is formulated as a Mixed-Integer Second-Order Cone Programming (MI-SOCP) model that minimizes the spatial residual between the target silhouette vector and the aggregate selected tile configurations. Let the grid dimensions after discretization be $H \times W$, yielding a total number of lattice cells $d = H \cdot W$. The target silhouette vector $\mathbf{g}$ resides in $\mathbb{R}^{d\times 1}$. Let k_n^o denote the number of feasible placements (options) in the sub-dictionary $\mathcal{D}_n$ for the n-th tile. The total cardinality of the global dictionary matrix $\mathbf{D} \in \mathbb{R}^{d\times k_o}$ is $k_o = \sum_{n=1}^{N} k_n^o$, constructed by horizontally stacking the sub-dictionaries:

$$\mathbf{D} = [\mathcal{D}_1 \mid \mathcal{D}_2 \mid \cdots \mid \mathcal{D}_N]\,. \tag{4}$$

We define a global binary selection vector $\mathbf{x}_s \in \{0, 1\}^{k_o\times 1}$, partitioned into sub-vectors $\mathbf{x}_s = [\mathbf{x}_1^\top, \ldots, \mathbf{x}_N^\top]^\top$, where each $\mathbf{x}_n \in \{0, 1\}^{k_n^o\times 1}$ is an indicator vector activating a specific configuration for tile n. Let $\mathbf{z} \in \mathbb{R}^{d\times 1}$ represent the reconstruction error vector, and let $q \in \mathbb{R}_{\geq 0}$ be a continuous optimization decision variable serving as a slack upper bound for the error norm. The complete MI-SOCP optimization framework is formulated as follows:

$$\begin{aligned}
\min_{\mathbf{x}_s,\mathbf{z},q} \quad & \lambda \cdot q \\
\text{s.t.} \quad & \mathbf{g} = \mathbf{D}\mathbf{x}_s + \mathbf{z} && (5)\\
& \|\mathbf{z}\|_2 \leq q && (6)\\
& \sum_{j=1}^{k_n^o} x_{n,j} = 1, \quad \forall n \in \{1, \ldots, N\} && (7)\\
& \mathbf{x}_s \in \{0, 1\}^{k_o}, \quad 0 \leq q \leq Q_{\max} && (8)\\
& -Z_{\max} \leq z_i \leq Z_{\max}, \quad \forall z_i \in \mathbf{z} && (9)
\end{aligned}$$

where $(\mathbf{x}_s, \mathbf{z}, q)$ is the joint decision tuple. Constraint (7) enforces configuration uniqueness per tile. The parameter λ scales the objective, while $Z_{\max}$ and $Q_{\max}$ bound the element-wise error and total error norm, respectively. In our benchmarks, we set $\lambda = 10^3$, $Z_{\max} = \max\{N, 10\} = 10$, and $Q_{\max} = 10^4$.

4.1 Comparative Benchmark and Performance Analysis

To rigorously evaluate the proposed MI-SOCP framework, a comparative study was conducted against Genetic Algorithm (GA) and Simulated Annealing (SA) baselines across 50 distinct Tangram instances of varying geometric complexity. All source code and experimental results are publicly available to facilitate reproducibility at [2].

Experimental Setup: Experiments were run on an Intel i7-8565U (1.80 GHz, 16 GB RAM) using MATLAB R2023b. Solvers were configured as follows:

- **MI-SOCP (Proposed):** Gurobi v12.0.3 [8] with MIPFocus = 1 to prioritize feasible solutions.
- **GA Baseline:** MATLAB `ga()` with scattered crossover and custom group-constrained discrete mutation.
- **SA Baseline:** MATLAB `simulannealbnd()` with exponential cooling and a bounds-adjusting fitness function.
- **Translation FTO:** Vertex matching followed by MATLAB `patternsearch()`, see details in Section 3.1.

All simulations were implemented in MATLAB, with performance-critical preprocessing accelerated via C++ MEX routines using the Clipper2 library[2]. To evaluate stochastic variance, 20 puzzles were tested across three random seeds. All frameworks enforced a strict 30-minute time limit per instance, with graphical rendering disabled to eliminate overhead. Table 1 details all parameters and Table 2 summarizes the single-run results; comprehensive multi-run results are available in our repository [2].

Success Criteria: Outputs from each optimization method undergo a translation fine-tuning optimization (FTO) stage to yield continuous final configurations. These configurations are evaluated using four key performance indicators (KPIs): (1) **Covered Area (CA):** the target area covered by the union of placed tiles; (2) **Uncovered Area (UCA):** the empty target area; (3) **Overlap Area (OLA):** the cumulative area of pairwise tile intersections; and (4) **Outside Area (OSA):** the tile area extending beyond the target boundary.

An instance is classified as a *Success* if the solver terminates within the 30-minute time limit and the post-FTO solution satisfies:

$$(\mathrm{CA} \geq \mathrm{CA}_{TH}) \ \wedge \ (\mathrm{UCA} < \mathrm{Tol}_{TH}) \ \wedge \ (\mathrm{OLA} < \mathrm{Tol}_{TH}) \ \wedge \ (\mathrm{OSA} < \mathrm{Tol}_{TH}) \quad (10)$$

[2] Implemented as MATLAB MEX functions using Clipper2 [10].

Table 1 Operational Parameters and Code Tolerances

Sol.	Parameter / Routine	Value	Sol.	Parameter / Routine	Value
MISOCP	`MIPFocus`	1	**GA**	`PopulationSize`	2,000
	`ImproveStartGap`	0.59		`mutation_prob`	0.3
	`Heuristics`	0.05*		`MaxTime`	1,800 s†
	`IntFeasTol`	10^{-6}		`MaxGenerations` / `Stall`	10^9
	`FeasibilityTol` / `OptimalityTol`	10^{-5}	**SA**	`InitialTemperature`	800
	`Presolve` / `PrePasses`	1 / 3		`MaxTime`	1,800 s†
	`MaxTime`	1,800 s†			
FTO	`MaxFunctionEvaluations`	10^4			
	`MeshTolerance`	10^{-4}			
	`MaxTime`	300 s			

*A default heuristic value of 0.05 was used for all reported results; however, increasing this value (e.g., to 0.1–0.4) can improve performance on a small subset of highly constrained puzzles (e.g., `shape35`).

†Uniform time limit of 1,800 seconds (30 minutes) enforced across all optimization frameworks, and 5 minutes for `patternsearch()`. The 30-minute limit applies to the core solver; the subsequent FTO stage has a separate 5-minute limit.

where CA_{TH} is the required coverage threshold. Given a total tile area of 784, this threshold is set to $\mathrm{CA}_{TH} = 783$, with an error tolerance of $\mathrm{Tol}_{TH} = 10^{-1}$.

Computational Results: Performance metrics across the 50 benchmark instances are detailed in Table 2, yielding two primary insights:

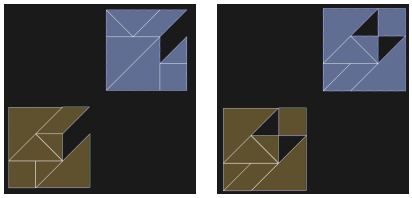
- **Success Rate:** The proposed MI-SOCP framework achieved a 98% success rate (49 out of 50 instances)[3] By providing accurate discrete flips, rotations, and close translations, it enabled the FTO to converge to near-exact solutions across nearly all targets. Conversely, meta-heuristics struggled with combinatorial complexity, with GA and SA solving only 4 (8%) and 8 (16%) instances, respectively.
- **Runtime Dynamics:** MI-SOCP demonstrated high efficiency: 40 instances (80%) converged in under 5 minutes, and 48 instances completed within 15 minutes, highlighting the power of the overall method. In contrast, most meta-heuristic runs exhausted the 30-minute time limit and ended up with failed convergence status.

4.2 MI-SOCP Qualitative Discussion

The qualitative performance of the MI-SOCP model is characterized by three primary structural factors:

1. **Alternative Optimal Solutions:** The solver frequently identified degenerate optima, distinct tile arrangements satisfying the target geometry via unique paths, demonstrating multi-modal layout characteristics.

[3] The single timeout instance for MISOCP (`shape35`) successfully converged within 60 minutes across three independent runs with different random seeds (See folder 'Shape35Run' in [2]).



(a) Degenerate optimum. (b) Internal void resolution.

Fig. 3 Topological complexity and discretization artifacts. Reference (azure); MI-SOCP (olive). The model isolates degenerate configurations and cleanly handles internal voids.

2. **Topological Robustness:** As shown in Figure 3, the model resolves targets containing non-simply connected topologies (internal voids) without requiring explicit boundary constraints, validating the fractional coverage approach.
3. **Discretization Fidelity:** Global geometric properties remain preserved despite discretization effects. Empirical tests indicate $ds = 1/28$ provides an ideal trade-off between fidelity and computational tractability.

5 Conclusion and Future Work

This paper presented an MI-SOCP framework that successfully bridges continuous geometric profiles with discrete mathematical optimization by mapping polygonal structures to fractional coverage vectors via the Sutherland-Hodgman algorithm. The proposed approach consistently outperformed meta-heuristic baselines, achieving a 98% success rate across all 50 benchmark targets. Notably, the framework efficiently isolated multi-modal degenerate configurations and cleanly resolved complex topological features, such as internal voids. Furthermore, the integration of a continuous translation fine-tuning optimization (FTO) stage acts as a crucial refinement and validation mechanism, while the subsequent geometric KPIs assess whether low-residual discrete approximations yield high quality continuous configurations.

Future work will explore multi-scale adaptive and non-uniform grid refinement schemes to further optimize computational efficiency. Additionally, we aim to extend this framework to 3D packing structures to address complex spatial optimization problems in manufacturing and biological applications.

Table 2 Detailed evaluation metrics across 50 benchmark problem instances (single-run per puzzle). Columns denote instance ID, name, Cartesian product of valid orientations, and performance profiles for each solver method: execution time (minutes), CA, UCA, OLA, OSA (see definitions in Section 4.1) and convergence status (S = Success and F = Failure, see Equation (10))

			SolverGA						**SolverSA**						**SolverMISOCP**					
ID	Name	Options	Time	CA	UCA	OLA	OSA	Stat	Time	CA	UCA	OLA	OSA	Stat	Time	CA	UCA	OLA	OSA	Stat
1	shape55	9.57×10^{17}	21.94	735	49.00	49.00	0.00	[F]	30.01	784	0.00	0.00	0.00	[S]	0.06	784	0.00	0.00	0.00	[S]
2	shape54	1.81×10^{17}	22.31	784	0.00	0.00	0.00	[S]	30.01	784	0.00	0.00	0.00	[S]	0.07	784	0.00	0.00	0.00	[S]
3	shape56	6.44×10^{17}	29.46	759	25.20	25.20	0.00	[F]	30.00	735	49.00	49.00	0.00	[F]	0.09	784	0.00	0.00	0.00	[S]
4	camel16	1.35×10^{20}	27.56	726	58.10	58.10	0.00	[F]	30.00	750	33.60	33.60	0.00	[F]	0.11	784	0.00	0.00	0.00	[S]
5	square1	6.05×10^{21}	19.91	755	29.10	28.60	0.42	[F]	30.00	743	40.70	40.70	1.31e-08	[F]	0.12	784	0.00	0.00	0.00	[S]
6	shape53	2.16×10^{19}	25.97	767	16.80	16.80	0.00	[F]	30.00	750	34.40	33.60	0.72	[F]	0.15	784	0.00	0.00	0.00	[S]
7	shape23	3.23×10^{19}	15.02	735	49.00	49.00	1.13e-03	[F]	30.01	757	26.70	26.70	0.00	[F]	0.15	784	0.00	0.00	0.00	[S]
8	shape29	3.38×10^{19}	25.39	768	16.30	16.30	0.00	[F]	30.00	767	16.80	16.80	1.31e-08	[F]	0.16	784	0.00	0.00	0.00	[S]
9	shape37	1.49×10^{21}	19.67	730	53.90	53.20	0.72	[F]	30.00	726	58.40	57.20	1.21	[F]	0.17	784	0.00	0.00	0.00	[S]
10	shape32	2.28×10^{21}	25.14	746	37.80	37.80	0.00	[F]	30.00	739	45.20	45.20	0.00	[F]	0.17	784	0.00	0.00	0.00	[S]
11	shape19	8.67×10^{19}	27.75	767	16.80	16.80	0.00	[F]	30.01	767	16.80	16.80	0.00	[F]	0.26	784	0.00	0.00	0.00	[S]
12	shape51	2.92×10^{21}	26.41	732	51.80	51.80	0.00	[F]	30.01	767	16.80	16.80	0.00	[F]	0.27	784	0.00	0.00	0.00	[S]
13	shape24	9.35×10^{18}	25.07	739	44.90	44.90	0.00	[F]	30.00	750	33.60	32.90	0.72	[F]	0.30	784	0.00	0.00	0.00	[S]
14	shape13	2.33×10^{18}	29.11	743	41.30	41.30	5.66e-04	[F]	30.00	784	0.00	0.00	0.00	[S]	0.45	784	0.00	0.00	0.00	[S]
15	shape22	5.45×10^{17}	14.53	776	8.41	8.41	0.00	[F]	30.00	767	16.80	16.80	0.00	[F]	0.46	784	0.00	0.00	0.00	[S]
16	shape10	1.33×10^{20}	29.54	767	16.80	16.80	0.00	[F]	30.01	776	8.41	8.41	0.00	[F]	0.50	784	0.00	0.00	0.00	[S]
17	shape25	1.21×10^{19}	22.65	776	8.41	8.41	0.00	[F]	30.00	767	16.80	16.80	0.00	[F]	0.54	784	0.00	0.00	0.00	[S]
18	shape44	1.68×10^{19}	7.59	750	33.60	25.20	8.41	[F]	30.00	736	47.60	47.60	0.00	[F]	0.54	784	0.00	0.00	0.00	[S]
19	shape18	2.83×10^{18}	7.85	720	63.90	53.30	10.60	[F]	30.05	784	0.00	0.00	0.00	[S]	0.64	784	0.00	0.00	0.00	[S]
20	shape50	1.37×10^{21}	26.97	747	37.20	37.20	0.00	[F]	30.01	744	39.50	39.50	0.00	[F]	0.67	784	0.00	0.00	0.00	[S]
21	shape58	6.11×10^{19}	17.94	738	45.80	45.80	3.20e-03	[F]	30.01	758	26.30	26.30	1.37e-02	[F]	0.69	784	5.66e-04	5.66e-04	0.00	[S]
22	shape30	1.36×10^{18}	17.03	784	0.00	0.00	0.00	[S]	30.00	784	0.00	0.00	0.00	[S]	0.76	784	0.00	0.00	0.00	[S]
23	shape2	5.05×10^{19}	28.68	734	50.40	50.40	0.00	[F]	30.00	735	48.80	45.10	3.70	[F]	0.80	784	0.00	0.00	0.00	[S]
24	shape17	1.55×10^{19}	28.31	722	61.60	60.70	0.88	[F]	30.00	759	25.20	16.80	8.41	[F]	0.96	784	0.00	0.00	0.00	[S]
25	shape20	5.37×10^{19}	24.14	759	24.90	24.90	0.00	[F]	30.00	767	16.80	16.80	0.00	[F]	0.98	784	0.00	0.00	0.00	[S]
26	shape31	9.56×10^{18}	22.19	750	33.60	33.60	0.00	[F]	30.00	767	16.80	16.80	0.00	[F]	1.02	784	0.00	0.00	0.00	[S]
27	shape62	2.05×10^{18}	24.15	784	0.00	0.00	0.00	[S]	30.00	784	0.00	0.00	0.00	[S]	1.24	784	0.00	0.00	0.00	[S]
28	shape8	3.86×10^{21}	24.57	758	25.90	25.90	3.72e-03	[F]	30.00	731	53.10	53.10	2.83e-03	[F]	1.51	784	0.00	0.00	0.00	[S]
29	shape21	2.16×10^{15}	20.95	784	0.00	0.00	0.00	[S]	30.01	784	0.00	0.00	0.00	[S]	1.83	784	0.00	0.00	0.00	[S]
30	shape46	1.41×10^{20}	19.80	766	17.60	17.50	1.68e-02	[F]	30.00	750	33.60	29.40	4.20	[F]	2.03	784	0.00	0.00	0.00	[S]
31	shape26	1.95×10^{19}	18.95	767	16.80	16.80	0.00	[F]	30.00	784	0.00	0.00	0.00	[S]	2.37	784	0.00	0.00	0.00	[S]
32	shape40	4.21×10^{19}	29.95	756	28.10	26.70	1.44	[F]	30.00	750	34.30	33.40	0.93	[F]	2.90	784	0.00	0.00	0.00	[S]
33	shape43	1.12×10^{21}	28.56	725	58.90	58.90	1.72e-04	[F]	30.00	764	20.30	19.80	0.50	[F]	3.03	784	0.00	0.00	0.00	[S]
34	shape60	1.67×10^{22}	19.32	716	68.10	40.40	27.70	[F]	30.01	752	32.20	23.80	8.41	[F]	3.34	784	0.00	0.00	0.00	[S]
35	shape45	5.49×10^{20}	20.60	749	35.10	35.10	0.00	[F]	30.00	726	57.80	50.90	6.90	[F]	3.63	784	0.00	0.00	0.00	[S]
36	shape47	8.87×10^{21}	21.06	737	46.70	9.30	37.40	[F]	30.01	755	28.70	8.41	20.30	[F]	3.82	784	0.00	0.00	0.00	[S]
37	shape42	2.02×10^{21}	26.63	753	30.70	20.30	10.40	[F]	30.00	741	42.90	42.00	0.93	[F]	3.88	784	0.00	0.00	0.00	[S]
38	shape36	1.30×10^{21}	26.71	767	16.80	16.80	0.00	[F]	30.00	739	45.00	44.70	0.25	[F]	4.03	784	1.60e-03	1.60e-03	3.27e-09	[S]
39	shape61	9.29×10^{19}	19.14	751	33.10	33.10	0.00	[F]	30.00	767	16.80	16.80	0.00	[F]	4.26	784	2.43e-03	2.43e-03	0.00	[S]
40	shape52	2.31×10^{21}	22.93	766	18.40	17.70	0.72	[F]	30.00	755	28.90	25.10	3.81	[F]	4.69	784	4.00e-03	4.00e-03	0.00	[S]
41	shape48	4.72×10^{21}	29.39	729	55.40	51.50	3.93	[F]	30.00	726	58.00	44.50	13.40	[F]	5.43	784	0.00	0.00	0.00	[S]
42	shape49	1.95×10^{21}	26.40	744	40.50	33.10	7.39	[F]	30.01	693	90.60	79.70	10.90	[F]	5.70	784	1.60e-03	1.60e-03	3.27e-09	[S]
43	shape59	5.20×10^{18}	25.58	747	36.50	23.90	12.60	[F]	30.00	759	24.50	24.50	0.00	[F]	5.91	784	0.00	0.00	0.00	[S]
44	shape41	8.62×10^{21}	27.48	764	19.70	17.60	2.13	[F]	30.00	747	37.40	30.10	7.26	[F]	6.55	784	0.00	0.00	0.00	[S]
45	shape39	7.03×10^{20}	29.97	759	25.20	25.20	0.00	[F]	30.01	752	32.20	32.20	1.93e-02	[F]	7.48	784	0.00	0.00	0.00	[S]
46	shape57	2.84×10^{20}	25.29	755	29.40	22.50	6.97	[F]	30.00	759	25.20	25.20	0.00	[F]	9.64	784	0.00	0.00	0.00	[S]
47	shape38	1.40×10^{21}	20.13	749	35.00	34.20	7.30e-01	[F]	30.00	749	35.30	34.30	1.00	[F]	11.50	784	2.09e-02	2.09e-02	1.60e-06	[S]
48	shape34	8.16×10^{20}	29.42	755	28.80	28.10	7.25e-01	[F]	30.00	767	16.80	16.80	0.00	[F]	13.13	784	1.21e-02	2.43e-03	9.67e-03	[S]
49	shape33	2.18×10^{21}	23.38	739	45.20	18.40	26.80	[F]	30.00	748	36.00	35.30	7.23e-01	[F]	30.05	784	8.07e-03	4.86e-03	3.20e-03	[S]
50	shape35	1.11×10^{21}	27.36	751	32.70	32.70	0.00	[F]	30.00	753	30.80	30.80	0.00	[F]	30.05	749	35.10	35.10	0.00	[F]

Note: The single MISOCP timeout (`shape35`) converges robustly across 3 independent runs, each initialized with a distinct random seed, when MaxTime is set to 60 minutes. Full results, including multi-run data and individual runs for `shape35`, are provided in [2].

Acknowledgements

The work was supported by the Horizon 2020 research and innovation programme for the Bio4Comp project under grant agreement number 732482, and by the Israel Science Foundation (Grant No. 190/19).

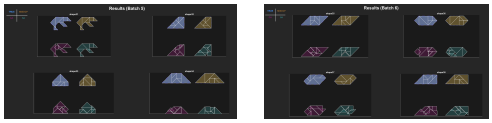


Fig. 4 Representative visual examples: Clockwise from top-left: Target (True) Profile, MI-SOCP result, SA result, and GA result. Full results provided in [2].

References

1. Bennell, J.A., Oliveira, J.F.: The geometry of nesting problems: A tutorial. European Journal of Operational Research **184**(2), 397–415 (2008)
2. Daniel, Y., Kugler, H.: Tangram-solver-github: A mixed-integer optimization framework for tangram puzzles. `https://github.com/yossidanphd18/TangramSolver` (2026). GitHub repository
3. Demaine, E.D., Demaine, M.L.: Jigsaw puzzles, edge matching, and polyomino packing: Connections and complexity. Graphs and Combinatorics **23**(1), 195–208 (2007)
4. Deutsch, E.S., Hayes Jr, K.C.: A heuristic solution to the tangram puzzle. Machine Intelligence **7**, 205–240 (1972)
5. Fasano, G.: A mixed integer programming formulation for the irregular packing problem. Discrete Applied Mathematics **163**, 19–33 (2014)
6. Gardner, M.: Mathematical games. Scientific American **231**(2), 98–103 (1974)
7. Golomb, S.: Polyominoes: Puzzles, Patterns, Problems, and Packings. Princeton University Press (1994)
8. Gurobi Optimization, LLC: Gurobi Optimizer Reference Manual (2024)
9. Ji, A., Kojima, N., Rush, N., Suhr, A., Vong, W.K., Hawkins, R., Artzi, Y.: Abstract visual reasoning with tangram shapes. In: Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing, pp. 582–601 (2022)
10. Johnson, A.: Clipper2: Polygon clipping and offset library. `https://github.com/AngusJohnson/Clipper2` (2024). GitHub repository
11. Karp, R.M.: Reducibility among combinatorial problems. In: Complexity of Computer Computations, pp. 85–103. Plenum Press (1972)
12. Knuth, D.E.: Dancing links. In: Millennial Perspectives in Computer Science, pp. 187–214 (2000)
13. Oflazer, K.: Solving tangram puzzles: A connectionist approach. International Journal of Intelligent Systems **8**(5), 603–616 (1993)
14. Pak, I.: Tiling simply connected regions with rectangles. J. Comb. Theory Ser. A **120**(7), 1563–1571 (2013)
15. Sutherland, I.E., Hodgman, G.W.: Reentrant polygon clipping. Communications of the ACM **17**(1), 32–42 (1974)
16. Wäscher, G., Haußner, H., Schumann, H.: An improved typology of cutting and packing problems. European Journal of Operational Research **183**(3), 1109–1130 (2007)
17. Yamada, F.M.: Generative approaches for solving tangram puzzles. Discover AI **2**(1), 23 (2024)
18. Yamada, F.M., Batagelo, H.C., Gois, J.P., Takahashi, H.: Tangan: solving tangram puzzles using generative adversarial network. Applied Intelligence **55**(7), 511 (2025)
19. Yamada, F.M., Gois, J.P., Batagelo, H.C.: Solving tangram puzzles using raster-based mathematical morphology. In: Proceedings of the 32nd SIBGRAPI Conference on Graphics, Patterns and Images (SIBGRAPI), pp. 88–95 (2019)
20. Zhao, Y., et al.: Learning from the tangram to solve mini visual tasks. arXiv preprint arXiv:2112.06113 (2021)